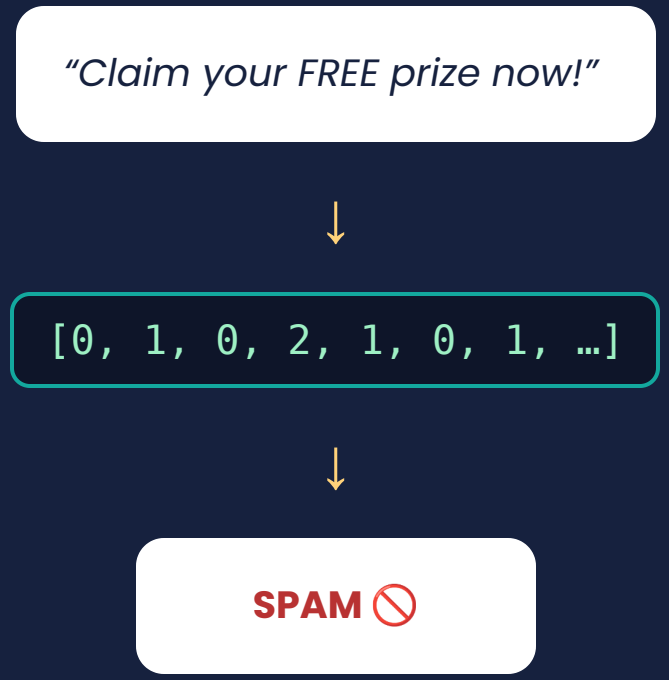
NATURAL LANGUAGE PROCESSING · LESSON 2

From Words to Numbers

Bag of Words, TF-IDF, and building your very first spam detector.



WARM-UP

Have you seen messages like these?

Have you ever received a message like one of these?

UNKNOWN NUMBER

CONGRATS! Your number won P50,000! Click the link now to claim your prize before it expires!

UNKNOWN NUMBER

Your bank account is LOCKED. Verify your details here within 24 hours or lose your money.

UNKNOWN NUMBER

Hi Ma, this is my new number. My phone broke. Please send load ASAP, I will explain later.

These are **spam and scam messages**. They try to trick people into giving away money or personal details.

WHY THIS MATTERS

Spam is a real-world problem



Scams cost real money

Many people lose their savings because a fake message looked real. Older people and young users are common targets.



Too many to check by hand

Billions of messages and emails are sent every day. No team of humans could read them all.



It is a big problem here too

In the Philippines, text scams became so common that a 2022 law required everyone to register their SIM cards.



Computers can help

Spam filters quietly check your messages and hide the dangerous ones. Today, you will build one yourself.

WHY THIS MATTERS

The big problem: computers only understand numbers

A computer cannot “read” the way we do. It can only do **math with numbers**. So before a computer can judge a message, we must turn the words into numbers.

What we see

“free prize now”

We instantly feel that this message is suspicious.

→

What the computer needs

```
[1, 0, 1, 0, 0, 1, 0]
```

A list of numbers that a math formula can work with.

This process is called **text representation** or **feature extraction**. It is the topic of today’s lesson.

WHY THIS MATTERS

Where do we use words-to-numbers?

Turning words into numbers is the first step of almost every NLP system you use.

Spam filters	Decide if a message is safe or dangerous.
Review sorting	Separate happy and angry reviews in online shops.
Search engines	Find the pages that best match the words you typed.
Recommendations	Suggest news articles or videos with similar topics.
AI chatbots	Turn your message into numbers before writing a reply.

If you understand today's lesson, you understand the **foundation** of all of these.

QUICK RECAP

Where we are in the NLP journey

In the last lesson, we learned how to **clean and organize** text.



Today we add two new steps: turning clean text into numbers, then using those numbers to make predictions.

LEARNING GOALS

Your mission today

🎯 **Today's mission:** build a program that reads a text message and decides if it is **spam** or **safe**.

1. Explain why computers need text turned into numbers.
2. Build a Bag of Words table by hand and with Python.
3. Explain how TF-IDF finds the important words.
4. Describe how a computer learns from examples.
5. Train and test a real spam detector.
6. Judge a model fairly and explain its limits.

ROADMAP

From a message to a prediction

This is the plan for building our spam detector.



Sections 1 and 2 cover Step 3. Section 3 covers Steps 4 and 5.

01

Bag of Words

The simplest way to turn text into numbers.

What is Bag of Words?

Bag of Words (BoW) turns text into numbers by **counting how many times each word appears**. It ignores the order of the words.

Think of a bag of candy 🍬

Pour the candy out and count each color: 3 red, 2 blue, 1 green. You do not care which one went in first. Bag of Words does the same with words.

Example

"free free prize now"

↓

`free: 2` `prize: 1` `now: 1`

Step 1: Build the vocabulary

Step 1: Collect every unique word from all the messages. This list is called the **vocabulary**.

Message 1: “win free money now”

Message 2: “free lunch at school now”

Message 3: “see you at school now”

Vocabulary (9 words, A to Z)

`at` `free` `lunch` `money` `now` `school` `see` `win` `you`

Each word gets its own column, like a spreadsheet.

Step 2: Count the words

Step 2: For each message, count how many times each vocabulary word appears.

Message	at	free	lunch	money	now	school	see	win	you
1. win free money now	0	1	0	1	1	0	0	1	0
2. free lunch at school now	1	1	1	0	1	1	0	0	0
3. see you at school now	1	0	0	0	1	1	1	0	1

Each **row** is now a list of numbers called a **vector**. Message 1 became `[0,1,0,1,1,0,0,1,0]`.

Every message now has the **same length**, even if the messages had different numbers of words.

SIMULATED DEMO

Bag of Words in Python

SIMULATED DEMO

```
from sklearn.feature_extraction.text import CountVectorizer

messages = ["win free money now",
            "free lunch at school now",
            "see you at school now"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(messages)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

OUTPUT

```
['at' 'free' 'lunch' 'money' 'now' 'school' 'see' 'win'
'you']
[[0 1 0 1 1 0 0 1 0]
 [1 1 1 0 1 1 0 0 0]
 [1 0 0 0 1 1 1 0 1]]
```

CountVectorizer comes from scikit-learn, a popular machine learning library.

fit means “learn the vocabulary.” **transform** means “count the words.”

The output matches the table we made by hand. Python just does it much faster.

BAG OF WORDS

Real text makes a very big table

Our real dataset

4,179 × 7,387

That is 4,179 training messages and 7,387 different words. The table has over 30 million cells.

But almost all are zeros

99.8%

A short text message uses only a few words, so most cells in its row are 0.

A table that is mostly zeros is called **sparse**. Python saves memory by storing only the non-zero numbers.

The weaknesses of Bag of Words

Word order is lost

“Dog bites man” and “man bites dog” get the exact same numbers, even though the news is very different.

Common words take over

Words like “the,” “you,” and “now” appear in almost every message. Their big counts can hide the useful words.

No sense of meaning

“Happy” and “glad” get separate columns. The computer does not know they mean almost the same thing.

The second problem has a smart fix. It is called **TF-IDF**.

02

TF-IDF

Finding the words that really matter.

TF-IDF

Not all words are equally important

Imagine a school photo

Everyone wears the same uniform. If I say “the student in the uniform,” you cannot find the person I mean.

But if I say “the student with **red shoes**,” you can find them right away. Rare details are more useful.

Words work the same way

A word like “now” appears in many messages, so it does not tell them apart.

A rarer word like “prize” or “money” gives a much stronger clue about what a message is about.

TF-IDF gives a high score to words that are **common in one message** but **rare across all messages**.

The two parts of TF-IDF

TF

Term Frequency

How often does the word appear in this message? A word repeated many times is probably important to that message.

`"free free prize" → TF of "free" is high`

IDF

Inverse Document Frequency

How rare is the word across all messages? A word found in only a few messages gets a high score. A word found everywhere gets a low score.

`"now" is everywhere → IDF is low`

TF-IDF = TF × IDF

IDF: measuring how rare a word is

Let's find the IDF score for three words from our three messages.

Word	Found in how many messages?	IDF score (rarity)	Meaning
`now`	3 of 3	`1.00`	Very common, weak clue
`school`	2 of 3	`1.29`	Somewhat common
`money`	1 of 3	**`1.69`**	Rare, strong clue

The rule: The fewer messages a word appears in, the higher its IDF score.

For the curious: scikit-learn uses IDF = log((1 + N) ÷ (1 + df)) + 1, where N is the number of messages and df is how many contain the word.

TF × IDF: the final importance score

Now we combine both parts. Here are the final TF-IDF scores for Message 1: “win free money now.”

Word	Bag of Words	TF-IDF
win	1	**0.58**
money	1	**0.58**
free	1	0.44
now	1	0.35

With Bag of Words, every word scored the same: 1.

With TF-IDF, **“win”** and **“money”** score highest because they only appear in this message.

“now” scores lowest because it appears in every message.

SIMULATED DEMO

TF-IDF in Python

SIMULATED DEMO

```
from sklearn.feature_extraction.text import (
    TfidfVectorizer)

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(messages)

# Show the scores rounded to 2 decimals
print(X.toarray().round(2))
```

OUTPUT

```
[[0.   0.44 0.   0.58 0.35 0.   0.   0.58 0.  ]
 [0.43 0.43 0.57 0.   0.34 0.43 0.   0.   0.  ]
 [0.41 0.   0.   0.   0.32 0.41 0.53 0.   0.53]]
```

The code is almost the same as before. We only swap CountVectorizer for **TfidfVectorizer**.

The columns are in vocabulary order. Look at the 5th column, **“now.”** It always has the lowest score in each row.

Each row is also scaled, so long and short messages can be compared fairly.

Bag of Words vs. TF-IDF

Feature	Bag of Words	TF-IDF
What it stores	How many times each word appears.	How important each word is.
Common words	Can get big, misleading counts.	Get lower scores automatically.
Numbers look like	Whole numbers: 0, 1, 2, 3…	Decimals: 0.35, 0.58…
Word order	Ignored.	Still ignored.
Best for	Simple, fast counting tasks.	Search engines and finding key words.

Neither one is always better. In Section 3, we will test both to see which works best for spam.

03

Building a Spam Detector

Teaching a computer to learn from examples.

MACHINE LEARNING

Rules vs. learning from examples

Old way: writing rules

"If a message has the word **free**, mark it as spam."

> *"Free lunch at school today!"*

This safe message gets blocked. Spammers also change their words to escape the rules.

New way: machine learning

We show the computer thousands of messages that are already labeled as **spam** or **ham** (safe).

The computer finds the patterns by itself. This is called **machine learning**.

Machine learning means **learning from examples** instead of following hand-written rules.

OUR DATA

Meet the dataset

Label	Message (shortened)
ham	Ok lar... Joking wif u oni...
spam	WINNER!! As a valued network customer you have been selected to receive a £900 prize...
ham	What's your room number again? Wanna make sure I'm knocking on the right door
spam	URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot!...

We will use the **SMS Spam Collection**, a free dataset of **5,572** real text messages.

4,825 are ham (86.6%) and **747** are spam (13.4%).

The messages come from the UK and Singapore. That is why you see slang like "lor" and "wif."

MACHINE LEARNING

Splitting the data: practice vs. exam

Training set: 75%

4,179 messages. These are like the practice questions you study before a test. The model learns from these.

Test set: 25%

1,393 messages. These are like the final exam. The model has never seen them before.

Why hide the test set? If you saw the exact exam questions while studying, a perfect score would not prove you really learned.

MACHINE LEARNING

How Naive Bayes thinks

We will use a model called **Naive Bayes**. It treats every word like a clue, the way a detective would.

During training

It counts how often each word appears in spam and in ham. For example, “prize” and “claim” show up a lot in spam. “Sorry” and “home” show up a lot in ham.

During prediction

It looks at the words in a new message and combines all the clues. Then it gives a chance (probability) that the message is spam.

It is called “naive” because it assumes each word is a separate clue. This is a simple idea, but it works surprisingly well for spam.

SIMULATED DEMO

Training the spam detector

SIMULATED DEMO

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# 1. Split: 75% practice, 25% exam
X_train, X_test, y_train, y_test = train_test_split(
    df["message"], df["label"],
    test_size=0.25, random_state=42, stratify=df["label"])

# 2. Turn words into numbers
vec = CountVectorizer()
X_train_num = vec.fit_transform(X_train)
X_test_num  = vec.transform(X_test)

# 3. Train, then 4. Predict
model = MultinomialNB().fit(X_train_num, y_train)
predictions = model.predict(X_test_num)
print("Accuracy:", accuracy_score(y_test, predictions))
```

OUTPUT

```
Accuracy: 0.9863603732950467
```

We use **fit_transform** on the training set, so the vocabulary comes only from the practice messages.

We use only **transform** on the test set. The model must handle the exam with what it already learned.

The result is 0.986. That means the model labeled **98.6%** of the test messages correctly.

TESTING THE MODEL

Is 98.6% really good?

Our spam detector

98.6%

It studied the words and learned real patterns.

A “lazy” model that always says ham

86.6%

It never catches a single spam message, but it still looks good.

Because most messages are ham, accuracy alone can fool us. We need to look closer at the **mistakes**.

TESTING THE MODEL

The confusion matrix

A **confusion matrix** shows exactly where the model was right and where it got confused.

	Predicted: HAM	Predicted: SPAM
Actually HAM	**1,204** Correct: safe message kept	**2** False alarm: safe message blocked
Actually SPAM	**17** Missed: spam got through	**170** Correct: spam caught

Green boxes are correct answers. Red boxes are mistakes.

The model caught **170 out of 187** spam messages. That is about **91%**.

Only **2** safe messages were wrongly blocked.

THINK ABOUT IT

Which mistake is worse?

False alarm

A real message is sent to the spam folder. Imagine missing a message from your teacher about tomorrow's exam.

Our model: 2 times

Missed spam

A scam message reaches your inbox. You might be tricked if you are not careful.

Our model: 17 times

💬 **Class discussion:** Which mistake is worse for a spam filter? Would your answer change for a bank or a hospital?

SIMULATED DEMO

Testing with our own messages

Let’s give the model brand-new messages that were not in the dataset.

New message	Prediction	Confidence
“Congratulations! You won a FREE phone. Call now to claim your prize!”	SPAM	above 99.9%
“Hey, are we still meeting at the library after class?”	HAM	above 99.9%
“URGENT! Your account has been selected for a cash reward. Text WIN to 80888”	SPAM	above 99.9%
“Free lunch at school today!”	HAM	99.9%

Notice the last one. Unlike the simple rule, the model was not fooled by the word “free.”

INSIDE THE MODEL

What did the model learn?

We can ask the model which words it thinks are the strongest clues.

Strongest spam clues

`claim` `prize` `150p` `tone` `500` `18` `guaranteed` `cs` `1000` `landline` `awarded` `uk`

Prizes, money, and phone services show up a lot in spam.

Strongest ham clues

`gt` `lt` `he` `lor` `da` `she` `later` `amp` `ll` `ask` `said` `home`

Everyday words like “later,” “ask,” and “home” show up in normal chats.

The clues “gt,” “lt,” and “amp” come from leftover web codes in the ham messages, such as `>` `<` and `&`.

EXPERIMENT

Bag of Words or TF-IDF? Let's test!

Machine learning engineers always test different options. Here are three versions tested on the same 1,393 exam messages.

Text to numbers	Model	Accuracy	Spam missed
Bag of Words	Naive Bayes	**98.6%**	17
TF-IDF	Naive Bayes	95.7%	60
TF-IDF	Linear SVM	**98.7%**	16

Surprise! TF-IDF with Naive Bayes did **worse**. A smarter-sounding method is not always better.

A different model, the **Linear SVM**, works well with TF-IDF. It learns a boundary line between spam and ham.

LIMITATIONS

Where our spam detector fails

Tricky spelling

“Fr33 c4sh pr1ze!! cl1ck n0w” fooled our model. It said **ham**, because it never saw these spellings.

Old and foreign data

Our data is old and comes from the UK and Singapore. It may not work well on Taglish scam texts.

No real understanding

The model only counts words. It does not understand meaning, so new scam tricks can slip through.

Real spam filters are **retrained often** with new examples, because scammers keep changing their tricks.

RESPONSIBLE AI

Building AI responsibly



Fairness

If a model is trained on one kind of language, it may work poorly for other groups of people. We should test it on many kinds of messages.



Privacy

Spam filters read people's private messages. Companies must protect this data and use it only to keep users safe.



Mistakes have costs

A blocked message could be an emergency text. People who build AI must think about who gets hurt by its errors.

Good AI engineers ask two questions: "Does it work?" and "Is it fair and safe?"

WHAT'S NEXT

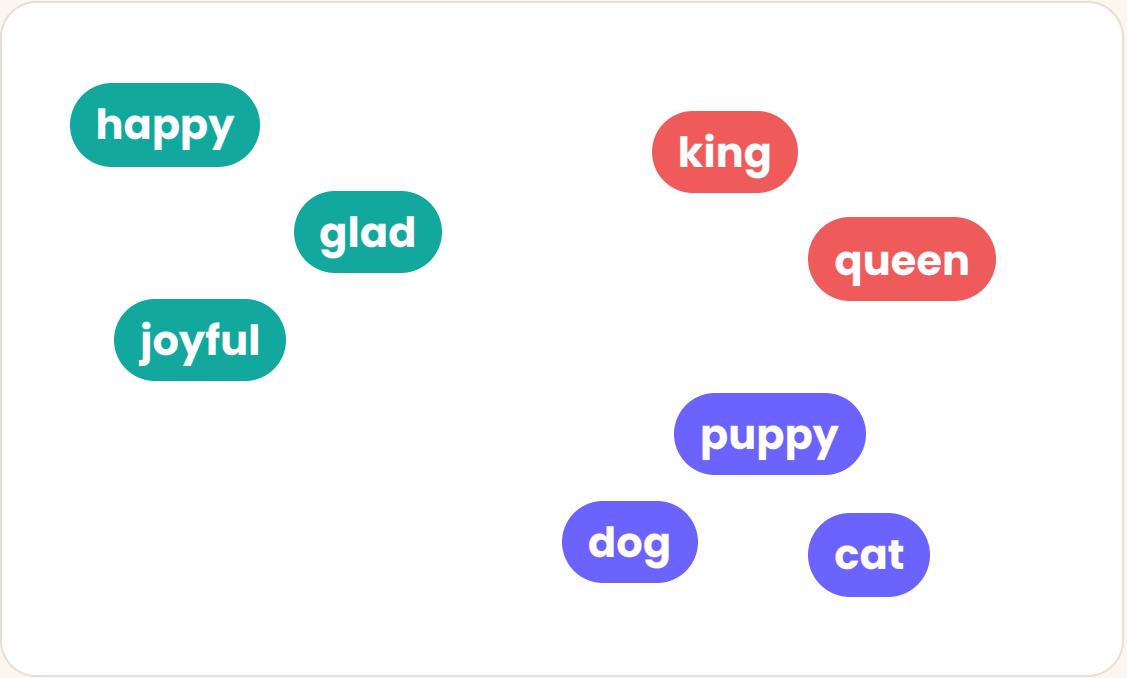
A sneak peek: word embeddings

Bag of Words and TF-IDF cannot tell that "happy" and "glad" mean almost the same thing. The next idea can.

Word embeddings turn each word into a point on a map.

Words with similar meanings end up **close together**.

Modern AI chatbots use a much more advanced version of this idea.



SUMMARY

Key takeaways

1. Computers can only work with numbers, so text must be turned into numbers first.
2. Bag of Words counts how many times each word appears.
3. TF-IDF scores words higher when they are rare across all messages.
4. Machine learning lets a computer learn patterns from labeled examples.
5. We test a model on data it has never seen, like a final exam.
6. Accuracy is not enough. Look at the kinds of mistakes the model makes.
7. Always experiment, and think about fairness and safety.

CHECK YOUR UNDERSTANDING

Quick quiz

Q1

Why do we turn words into numbers?

A. To make text shorter
B. Because computers can only do math with numbers
C. To remove stop words

Q2

What does Bag of Words count?

A. Letters in each word
B. How many times each word appears
C. The number of sentences

Q3

In TF-IDF, a word found in every message gets…

A. A low score
B. A high score
C. Removed completely

Q4

Why do we keep a separate test set?

A. To check the model on messages it never saw
B. To make training faster
C. To store extra spam

Q5

A model always says “ham” and gets 86.6% accuracy. What does this show?

A. It is a great model
B. Accuracy alone can be misleading
C. Spam is rare, so we can ignore it

CHECK YOUR UNDERSTANDING

Quiz answers

Q1

Answer: B

A model is a math formula, so it needs numbers.

Q2

Answer: B

It counts each word and ignores word order.

Q3

Answer: A

Common words are weak clues, so they get low scores.

Q4

Answer: A

Like a final exam, it shows if the model really learned.

Q5

Answer: B

It catches zero spam. We must also check the mistakes.

ACTIVITY

Try it yourself

Activity 1: Be the vectorizer

Make a Bag of Words table by hand for these three messages:

“buy cheap load now”
“see you at the mall now”
“cheap shoes at the mall now”

Which word would have the lowest IDF score?

Activity 2: Beat the detector

In groups, write a spam message that you think could fool our model. Then write a safe message that might be wrongly marked as spam.

Test them in the notebook. Explain why the model made each choice.

Bonus: How would you make the spam detector work better for messages in Filipino or Taglish?

NATURAL LANGUAGE PROCESSING

Thank you for listening!

You turned words into numbers and built a real AI model today.
What questions do you have?

`words` `→` `numbers` `→` `predictions` `!`